

Section 11.5 - Machine translation

January 2, 2026

1 Machine translation

1.1 An RNN model

1.1.1 Parse the dataset

- Translation from English to Spanish
- A line = ('English tokens', '[start] Spanish tokens [end]')
- '[start]' and '[end]' are special tokens

```
[1]: text_file = "spa-eng/spa.txt"
with open(text_file) as f:
    lines = f.read().split("\n")[:-1]
text_pairs = []
for line in lines:
    english, spanish = line.split("\t")
    spanish = "[start] " + spanish + " [end]"
    text_pairs.append((english, spanish))
```

```
[2]: # Inspect a line of the dataset
import random

print(random.choice(text_pairs))
```

('My mother makes the best cakes in the world.', '[start] Mi madre hace las mejores tortas del mundo. [end]')

1.1.2 Shuffle & split the dataset

- 70% for training
- 15% for validation
- 15% for testing

```
[3]: import random

random.shuffle(text_pairs)
num_val_samples = int(0.15 * len(text_pairs))
num_train_samples = len(text_pairs) - 2 * num_val_samples
train_pairs = text_pairs[:num_train_samples]
val_pairs = text_pairs[num_train_samples:num_train_samples + num_val_samples]
```

```
test_pairs = text_pairs[num_train_samples + num_val_samples:]
```

1.1.3 TextVectorization

- Do separately for the English and the Spanish parts

```
[4]: import tensorflow as tf
import string
import re
from tensorflow import keras
from tensorflow.keras import layers

strip_chars = string.punctuation + "¿"
strip_chars = strip_chars.replace("[", "")
strip_chars = strip_chars.replace("]", "")

def custom_standardization(input_string):
    lowercase = tf.strings.lower(input_string)
    return tf.strings.regex_replace(
        lowercase, f"[{re.escape(strip_chars)}]", "")

vocab_size = 15000
sequence_length = 20

source_vectorization = layers.TextVectorization(
    max_tokens=vocab_size,
    output_mode="int",
    output_sequence_length=sequence_length,
)
target_vectorization = layers.TextVectorization(
    max_tokens=vocab_size,
    output_mode="int",
    output_sequence_length=sequence_length + 1,
    standardize=custom_standardization,
)
train_english_texts = [pair[0] for pair in train_pairs]
train_spanish_texts = [pair[1] for pair in train_pairs]
source_vectorization.adapt(train_english_texts)
target_vectorization.adapt(train_spanish_texts)
```

1.1.4 Split data for training, validation and testing

- What does a sample look like?
- 1) Source sequence
 - 2) [start] + Target sequence
 - 3) Target sequence + [ending]

```
[5]: batch_size = 64

def format_dataset(eng, spa):
    eng = source_vectorization(eng)
    spa = target_vectorization(spa)
    return ({
        "english": eng,
        "spanish": spa[:, :-1],
    }, spa[:, 1:])

def make_dataset(pairs):
    eng_texts, spa_texts = zip(*pairs)
    eng_texts = list(eng_texts)
    spa_texts = list(spa_texts)
    dataset = tf.data.Dataset.from_tensor_slices((eng_texts, spa_texts))
    dataset = dataset.batch(batch_size)
    dataset = dataset.map(format_dataset, num_parallel_calls=4)
    return dataset.shuffle(2048).prefetch(16).cache()

train_ds = make_dataset(train_pairs)
val_ds = make_dataset(val_pairs)
test_ds = make_dataset(test_pairs)
```

```
[6]: # Inspect the batch shape
for inputs, targets in train_ds.take(1):
    print(f"inputs['english'].shape: {inputs['english'].shape}")
    print(f"inputs['spanish'].shape: {inputs['spanish'].shape}")
    print(f"targets.shape: {targets.shape}")
```

```
inputs['english'].shape: (64, 20)
inputs['spanish'].shape: (64, 20)
targets.shape: (64, 20)
```

1.1.5 GRU-based encoder

```
[7]: from tensorflow import keras
from tensorflow.keras import layers

embed_dim = 256
latent_dim = 1024

source = keras.Input(shape=(None,), dtype="int64", name="english")
x = layers.Embedding(vocab_size, embed_dim, mask_zero=True)(source)
encoded_source = layers.Bidirectional(
    layers.GRU(latent_dim), merge_mode="sum")(x)
```

1.1.6 GRU-based decoder and the end-to-end model

- Other metrics instead of Accuracy?

```
[8]: past_target = keras.Input(shape=(None,), dtype="int64", name="spanish")
x = layers.Embedding(vocab_size, embed_dim, mask_zero=True)(past_target)
decoder_gru = layers.GRU(latent_dim, return_sequences=True)
x = decoder_gru(x, initial_state=encoded_source)
x = layers.Dropout(0.5)(x)
target_next_step = layers.Dense(vocab_size, activation="softmax")(x)
seq2seq_rnn = keras.Model([source, past_target], target_next_step)

seq2seq_rnn.compile(
    optimizer="rmsprop",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
```

1.1.7 Training the model

```
[9]: # Hide the warning messages
import logging

tf.get_logger().setLevel(logging.ERROR)

import warnings
warnings.filterwarnings(
    "ignore",
    message="You are saving your model as an HDF5 file.*"
)

callbacks = [
    keras.callbacks.ModelCheckpoint("translation_with_RNN.h5",
                                    save_best_only=True,
                                    save_weights_only=True,
                                    monitor="val_loss"
                                )
]

history = seq2seq_rnn.fit(train_ds, epochs=15, validation_data=val_ds,
    ↪callbacks=callbacks)
```

Epoch 1/15

1302/1302 [=====] - 97s 68ms/step - loss: 4.6827 -
accuracy: 0.3186 - val_loss: 3.8879 - val_accuracy: 0.3905

Epoch 2/15

1302/1302 [=====] - 78s 60ms/step - loss: 3.7139 -
accuracy: 0.4161 - val_loss: 3.2304 - val_accuracy: 0.4731

Epoch 3/15

1302/1302 [=====] - 78s 60ms/step - loss: 3.1963 -

```

accuracy: 0.4743 - val_loss: 2.8535 - val_accuracy: 0.5213
Epoch 4/15
1302/1302 [=====] - 79s 60ms/step - loss: 2.8373 -
accuracy: 0.5160 - val_loss: 2.6053 - val_accuracy: 0.5553
Epoch 5/15
1302/1302 [=====] - 79s 61ms/step - loss: 2.5621 -
accuracy: 0.5494 - val_loss: 2.4280 - val_accuracy: 0.5813
Epoch 6/15
1302/1302 [=====] - 79s 61ms/step - loss: 2.3390 -
accuracy: 0.5776 - val_loss: 2.3068 - val_accuracy: 0.5986
Epoch 7/15
1302/1302 [=====] - 79s 61ms/step - loss: 2.1570 -
accuracy: 0.6012 - val_loss: 2.2149 - val_accuracy: 0.6129
Epoch 8/15
1302/1302 [=====] - 79s 61ms/step - loss: 1.9986 -
accuracy: 0.6222 - val_loss: 2.1399 - val_accuracy: 0.6249
Epoch 9/15
1302/1302 [=====] - 79s 61ms/step - loss: 1.8625 -
accuracy: 0.6410 - val_loss: 2.0864 - val_accuracy: 0.6326
Epoch 10/15
1302/1302 [=====] - 80s 61ms/step - loss: 1.7431 -
accuracy: 0.6569 - val_loss: 2.0493 - val_accuracy: 0.6402
Epoch 11/15
1302/1302 [=====] - 79s 61ms/step - loss: 1.6392 -
accuracy: 0.6715 - val_loss: 2.0090 - val_accuracy: 0.6461
Epoch 12/15
1302/1302 [=====] - 79s 61ms/step - loss: 1.5455 -
accuracy: 0.6851 - val_loss: 1.9837 - val_accuracy: 0.6506
Epoch 13/15
1302/1302 [=====] - 79s 61ms/step - loss: 1.4645 -
accuracy: 0.6966 - val_loss: 1.9631 - val_accuracy: 0.6539
Epoch 14/15
1302/1302 [=====] - 79s 61ms/step - loss: 1.3944 -
accuracy: 0.7067 - val_loss: 1.9514 - val_accuracy: 0.6556
Epoch 15/15
1302/1302 [=====] - 79s 61ms/step - loss: 1.3314 -
accuracy: 0.7160 - val_loss: 1.9378 - val_accuracy: 0.6578

```

1.1.8 Plotting the results

```

[10]: import matplotlib.pyplot as plt

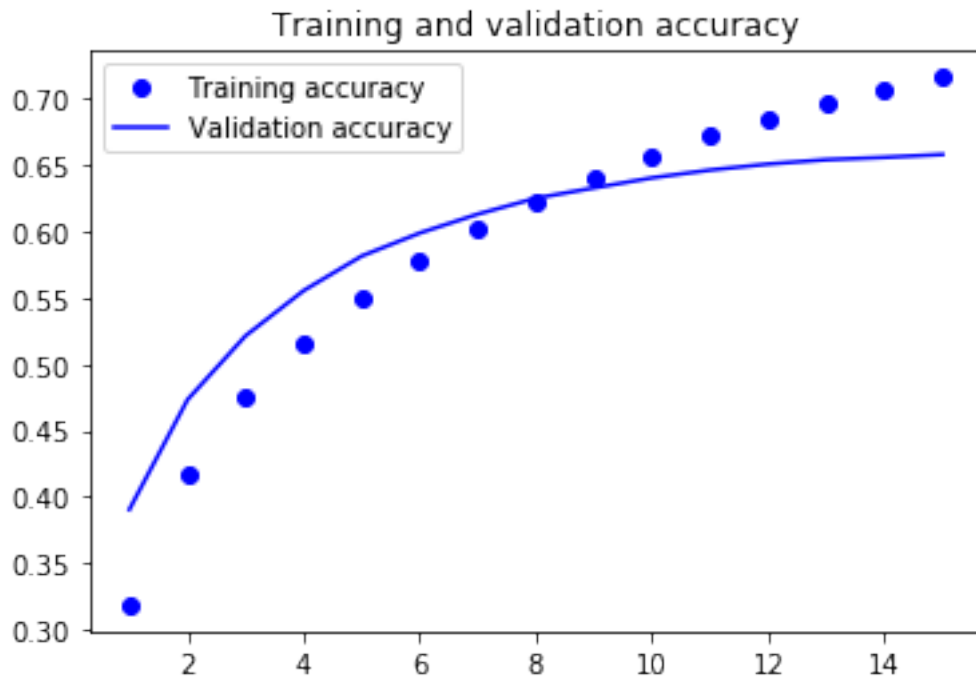
acc = history.history["accuracy"]
val_acc = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(acc) + 1)

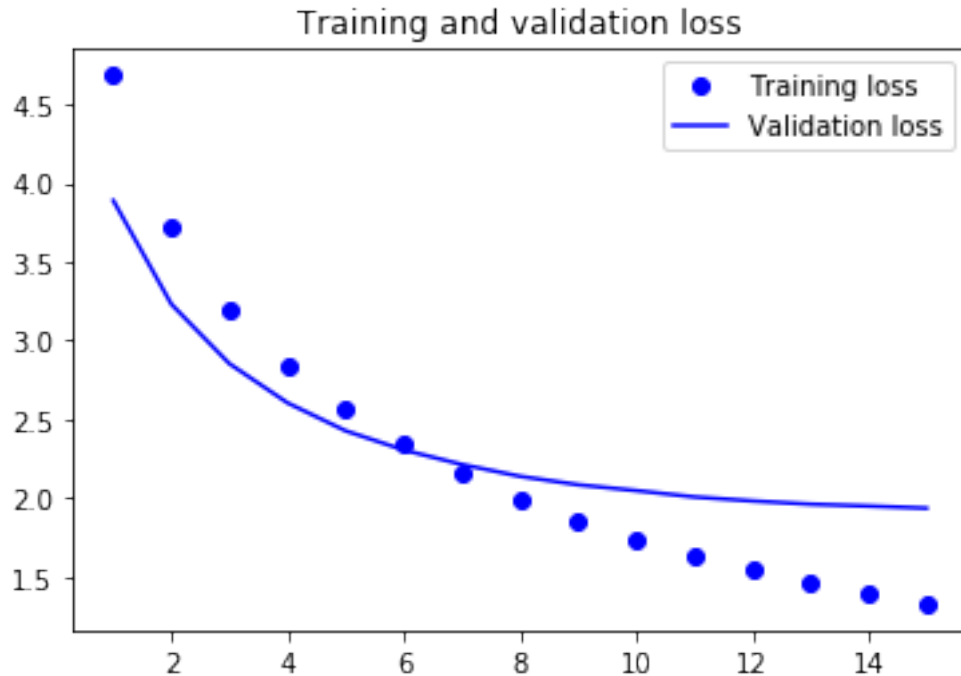
```

```

plt.plot(epochs, acc, "bo", label="Training accuracy")
plt.plot(epochs, val_acc, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()

```





1.1.9 Evaluating on the test set

- Test accuracy: 0.66

```
[11]: model = keras.Model([source, past_target], target_next_step)
```

```
model.compile(
    optimizer="rmsprop",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
```

```
model.load_weights("translation_with_RNN.h5")
```

```
print(f"Test acc: {model.evaluate(test_ds)[1]:.3f}")
```

```
279/279 [=====] - 13s 29ms/step - loss: 1.9253 -
accuracy: 0.6603
Test acc: 0.660
```

1.1.10 Translating new sentences with RNN encoder and decoder

- 1) RNN encoder
- 2) RNN decoder

```
[12]: import numpy as np
spa_vocab = target_vectorization.get_vocabulary()
```

```

spa_index_lookup = dict(zip(range(len(spa_vocab)), spa_vocab))
max_decoded_sentence_length = 20

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = model.predict(
            [tokenized_input_sentence, tokenized_target_sentence], verbose=0)
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

test_eng_texts = [pair[0] for pair in test_pairs]
for _ in range(20):
    input_sentence = random.choice(test_eng_texts)
    print("-")
    print(input_sentence)
    print(decode_sequence(input_sentence))

```

```

-
Tom couldn't make himself heard.
[start] tom no pudo hacer nada [end]
-
I'll have to find a part-time job.
[start] tendré que hacer un trabajo más trabajo [end]
-
The boy standing by the door is my brother.
[start] el chico que está parado en la puerta es mi hermano [end]
-
Over three thousand people attended the concert.
[start] tres horas tres personas [UNK] la semana [end]
-
Tom can speak French as well.
[start] tom puede hablar francés como bien [end]
-
I thought so at first.
[start] pensé que estaba en el primer [end]
-
She finished her homework in an hour.
[start] ella terminó la tarea en una hora [end]
-
What are you not telling me?
[start] qué te me dices no [end]

```


-
Do come in.
[start] ¡pásele [end]
-
The pond is very deep.
[start] el edificio es muy [UNK] [end]
-
Do I have to answer in English?
[start] tengo que cambiar en inglés [end]
-
Excuse me for interrupting you.
[start] perdone te [UNK] [end]
-
I know it's not Tom's fault.
[start] sé que no es culpa de tom [end]
-
Lock the door when you go out.
[start] cierra la puerta al salir [end]
-
You told her that you had finished the work three days before.
[start] le dijiste que la gente había terminado un par de días antes [end]
-
Where did you get the money to buy that dress?
[start] dónde le [UNK] para el dinero [end]
-
She's two years younger than him.
[start] ella es dos años más que ella [end]
-
The conference will take place in Tokyo.
[start] la reunión se [UNK] en tokio [end]
-
Can you speak a little slower?
[start] puedes hablar un poco más despacio [end]
-
Move along now.
[start] ahora mismo [end]

1.2 A Transformer model

1.2.1 The Transformer encoder

```
[13]: class TransformerEncoder(layers.Layer):
    def __init__(self, embed_dim, dense_dim, num_heads, **kwargs):
        super().__init__(**kwargs)
        self.embed_dim = embed_dim
        self.dense_dim = dense_dim
        self.num_heads = num_heads
        self.attention = layers.MultiHeadAttention(
```

```

        num_heads=num_heads, key_dim=embed_dim)
self.dense_proj = keras.Sequential(
    [layers.Dense(dense_dim, activation="relu"),
     layers.Dense(embed_dim),]
)
self.layernorm_1 = layers.LayerNormalization()
self.layernorm_2 = layers.LayerNormalization()

def call(self, inputs, mask=None):
    if mask is not None:
        mask = mask[:, tf.newaxis, :]
    attention_output = self.attention(
        inputs, inputs, attention_mask=mask)
    proj_input = self.layernorm_1(inputs + attention_output)
    proj_output = self.dense_proj(proj_input)
    return self.layernorm_2(proj_input + proj_output)

def get_config(self):
    config = super().get_config()
    config.update({
        "embed_dim": self.embed_dim,
        "num_heads": self.num_heads,
        "dense_dim": self.dense_dim,
    })
    return config

@classmethod
def from_config(cls, config):
    return cls(**config)

```

1.2.2 The Transformer decoder

```

[14]: class TransformerDecoder(layers.Layer):
    def __init__(self, embed_dim, dense_dim, num_heads, **kwargs):
        super().__init__(**kwargs)
        self.embed_dim = embed_dim
        self.dense_dim = dense_dim
        self.num_heads = num_heads
        self.attention_1 = layers.MultiHeadAttention(
            num_heads=num_heads, key_dim=embed_dim)
        self.attention_2 = layers.MultiHeadAttention(
            num_heads=num_heads, key_dim=embed_dim)
        self.dense_proj = keras.Sequential(
            [layers.Dense(dense_dim, activation="relu"),
             layers.Dense(embed_dim),]
        )
        self.layernorm_1 = layers.LayerNormalization()

```

```

self.layernorm_2 = layers.LayerNormalization()
self.layernorm_3 = layers.LayerNormalization()
self.supports_masking = True

def get_config(self):
    config = super().get_config()
    config.update({
        "embed_dim": self.embed_dim,
        "num_heads": self.num_heads,
        "dense_dim": self.dense_dim,
    })
    return config

def get_causal_attention_mask(self, inputs):
    input_shape = tf.shape(inputs)
    batch_size, sequence_length = input_shape[0], input_shape[1]
    i = tf.range(sequence_length)[:, tf.newaxis]
    j = tf.range(sequence_length)
    mask = tf.cast(i >= j, dtype="int32")
    mask = tf.reshape(mask, (1, input_shape[1], input_shape[1]))
    mult = tf.concat(
        [tf.expand_dims(batch_size, -1),
         tf.constant([1, 1], dtype=tf.int32)], axis=0)
    return tf.tile(mask, mult)

def call(self, inputs, encoder_outputs, mask=None):
    causal_mask = self.get_causal_attention_mask(inputs)
    if mask is not None:
        padding_mask = tf.cast(
            mask[:, tf.newaxis, :], dtype="int32")
        padding_mask = tf.minimum(padding_mask, causal_mask)
    else:
        padding_mask = mask
    attention_output_1 = self.attention_1(
        query=inputs,
        value=inputs,
        key=inputs,
        attention_mask=causal_mask)
    attention_output_1 = self.layernorm_1(inputs + attention_output_1)
    attention_output_2 = self.attention_2(
        query=attention_output_1,
        value=encoder_outputs,
        key=encoder_outputs,
        attention_mask=padding_mask,
    )
    attention_output_2 = self.layernorm_2(
        attention_output_1 + attention_output_2)

```

```

        proj_output = self.dense_proj(attention_output_2)
        return self.layernorm_3(attention_output_2 + proj_output)

    @classmethod
    def from_config(cls, config):
        return cls(**config)

```

1.2.3 The PositionalEmbedding layer

```

[15]: class PositionalEmbedding(layers.Layer):
    def __init__(self, sequence_length, input_dim, output_dim, **kwargs):
        super().__init__(**kwargs)
        self.token_embeddings = layers.Embedding(
            input_dim=input_dim, output_dim=output_dim)
        self.position_embeddings = layers.Embedding(
            input_dim=sequence_length, output_dim=output_dim)
        self.sequence_length = sequence_length
        self.input_dim = input_dim
        self.output_dim = output_dim

    def call(self, inputs):
        length = tf.shape(inputs)[-1]
        positions = tf.range(start=0, limit=length, delta=1)
        embedded_tokens = self.token_embeddings(inputs)
        embedded_positions = self.position_embeddings(positions)
        return embedded_tokens + embedded_positions

    def compute_mask(self, inputs, mask=None):
        return tf.math.not_equal(inputs, 0)

    def get_config(self):
        config = super().get_config() # Ensure superclass config is retrieved,
        ↪ properly
        config.update({
            "sequence_length": self.sequence_length,
            "input_dim": self.input_dim,
            "output_dim": self.output_dim,
        })
        return config

    @classmethod
    def from_config(cls, config):
        return cls(**config)

```

1.2.4 A Transformer model

```
[16]: embed_dim = 256
dense_dim = 2048
num_heads = 8

encoder_inputs = keras.Input(shape=(None,), dtype="int64", name="english")
x = PositionalEmbedding(sequence_length, vocab_size, embed_dim,
    ↪name="positional_embedding")(encoder_inputs)
encoder_outputs = TransformerEncoder(embed_dim, dense_dim, num_heads,
    ↪name="transformer_encoder")(x)

decoder_inputs = keras.Input(shape=(None,), dtype="int64", name="spanish")
x = PositionalEmbedding(sequence_length, vocab_size, embed_dim,
    ↪name="positional_embedding_decoder")(decoder_inputs)
x = TransformerDecoder(embed_dim, dense_dim, num_heads,
    ↪name="transformer_decoder")(x, encoder_outputs)
x = layers.Dropout(0.5, name="dropout_layer")(x)

decoder_outputs = layers.Dense(vocab_size, activation="softmax",
    ↪name="output_dense")(x)

transformer = keras.Model([encoder_inputs, decoder_inputs], decoder_outputs)

transformer.compile(
    optimizer="rmsprop",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
```

1.2.5 Training the Transformer model

```
[17]: history = transformer.fit(train_ds, epochs=30, validation_data=val_ds)

Epoch 1/30
1302/1302 [=====] - 83s 58ms/step - loss: 3.7878 -
accuracy: 0.4417 - val_loss: 2.8760 - val_accuracy: 0.5370
Epoch 2/30
1302/1302 [=====] - 65s 50ms/step - loss: 2.8418 -
accuracy: 0.5507 - val_loss: 2.5247 - val_accuracy: 0.5896
Epoch 3/30
1302/1302 [=====] - 65s 50ms/step - loss: 2.5475 -
accuracy: 0.5946 - val_loss: 2.4043 - val_accuracy: 0.6081
Epoch 4/30
1302/1302 [=====] - 65s 50ms/step - loss: 2.3886 -
accuracy: 0.6209 - val_loss: 2.3446 - val_accuracy: 0.6248
Epoch 5/30
1302/1302 [=====] - 65s 50ms/step - loss: 2.2839 -
accuracy: 0.6385 - val_loss: 2.3130 - val_accuracy: 0.6332
```

Epoch 6/30
1302/1302 [=====] - 65s 50ms/step - loss: 2.2121 - accuracy: 0.6518 - val_loss: 2.2998 - val_accuracy: 0.6376

Epoch 7/30
1302/1302 [=====] - 65s 50ms/step - loss: 2.1522 - accuracy: 0.6640 - val_loss: 2.3136 - val_accuracy: 0.6407

Epoch 8/30
1302/1302 [=====] - 66s 50ms/step - loss: 2.0893 - accuracy: 0.6760 - val_loss: 2.2761 - val_accuracy: 0.6471

Epoch 9/30
1302/1302 [=====] - 65s 50ms/step - loss: 2.0346 - accuracy: 0.6865 - val_loss: 2.2365 - val_accuracy: 0.6570

Epoch 10/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.9866 - accuracy: 0.6951 - val_loss: 2.2370 - val_accuracy: 0.6610

Epoch 11/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.9473 - accuracy: 0.7020 - val_loss: 2.2431 - val_accuracy: 0.6643

Epoch 12/30
1302/1302 [=====] - 65s 50ms/step - loss: 1.9165 - accuracy: 0.7080 - val_loss: 2.2477 - val_accuracy: 0.6637

Epoch 13/30
1302/1302 [=====] - 65s 50ms/step - loss: 1.8874 - accuracy: 0.7132 - val_loss: 2.2587 - val_accuracy: 0.6612

Epoch 14/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.8630 - accuracy: 0.7180 - val_loss: 2.2762 - val_accuracy: 0.6654

Epoch 15/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.8442 - accuracy: 0.7216 - val_loss: 2.2791 - val_accuracy: 0.6653

Epoch 16/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.8231 - accuracy: 0.7254 - val_loss: 2.3055 - val_accuracy: 0.6644

Epoch 17/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.8053 - accuracy: 0.7289 - val_loss: 2.3145 - val_accuracy: 0.6640

Epoch 18/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.7900 - accuracy: 0.7316 - val_loss: 2.3379 - val_accuracy: 0.6588

Epoch 19/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.7727 - accuracy: 0.7347 - val_loss: 2.3415 - val_accuracy: 0.6658

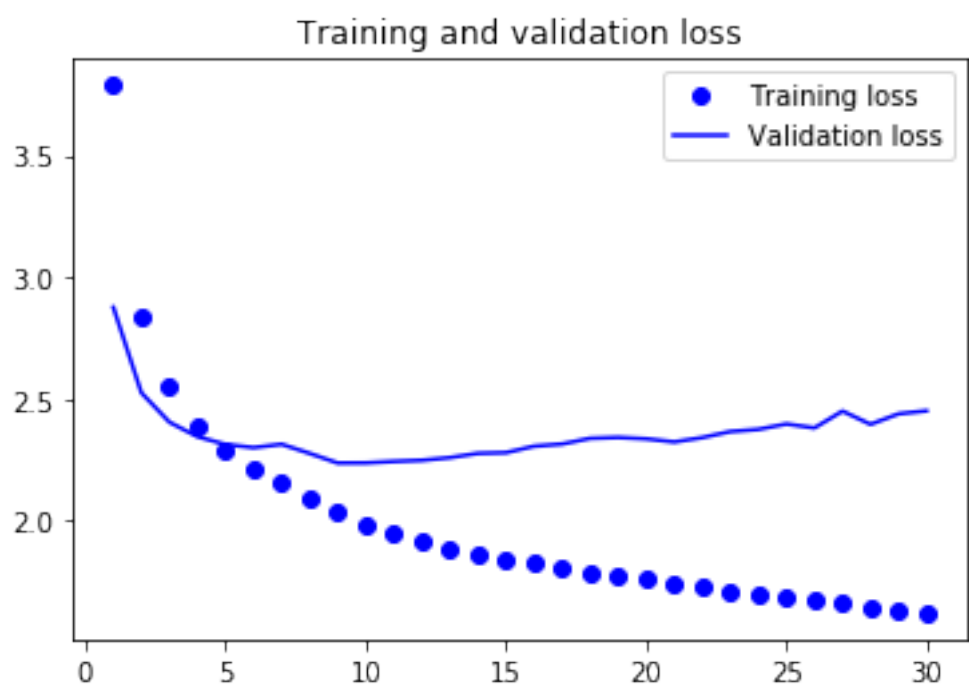
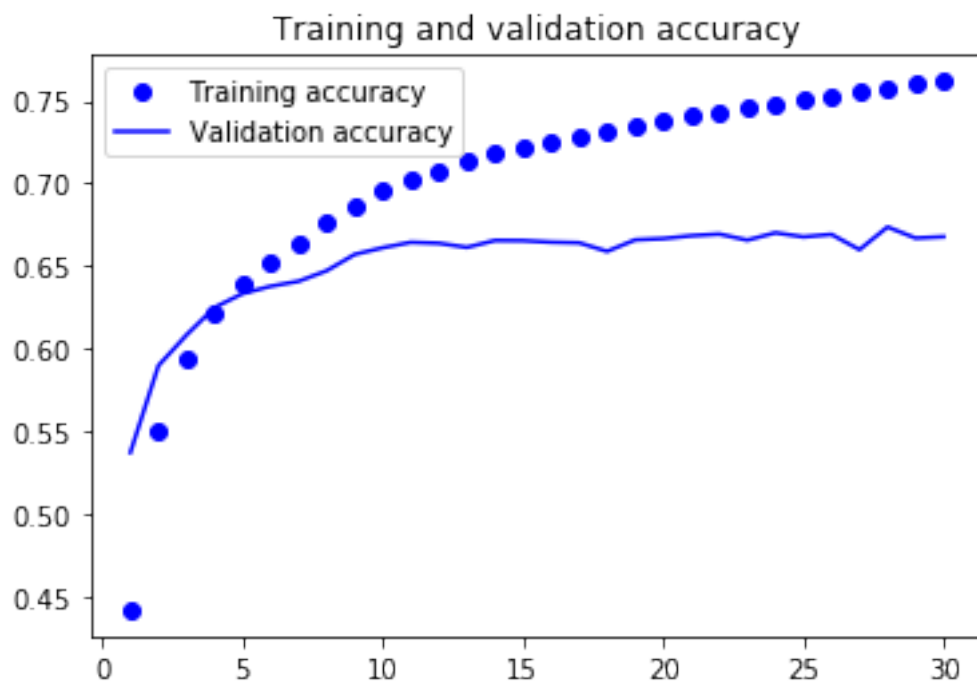
Epoch 20/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.7576 - accuracy: 0.7373 - val_loss: 2.3359 - val_accuracy: 0.6665

Epoch 21/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.7412 - accuracy: 0.7404 - val_loss: 2.3231 - val_accuracy: 0.6683

Epoch 22/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.7251 - accuracy: 0.7433 - val_loss: 2.3409 - val_accuracy: 0.6693
Epoch 23/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.7119 - accuracy: 0.7458 - val_loss: 2.3657 - val_accuracy: 0.6656
Epoch 24/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.7002 - accuracy: 0.7479 - val_loss: 2.3747 - val_accuracy: 0.6700
Epoch 25/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.6855 - accuracy: 0.7510 - val_loss: 2.3968 - val_accuracy: 0.6676
Epoch 26/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.6713 - accuracy: 0.7527 - val_loss: 2.3805 - val_accuracy: 0.6691
Epoch 27/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.6591 - accuracy: 0.7552 - val_loss: 2.4497 - val_accuracy: 0.6598
Epoch 28/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.6443 - accuracy: 0.7577 - val_loss: 2.3954 - val_accuracy: 0.6736
Epoch 29/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.6339 - accuracy: 0.7599 - val_loss: 2.4385 - val_accuracy: 0.6668
Epoch 30/30
1302/1302 [=====] - 66s 50ms/step - loss: 1.6211 - accuracy: 0.7618 - val_loss: 2.4508 - val_accuracy: 0.6677

1.2.6 Plotting the results

```
[18]: acc = history.history["accuracy"]
val_acc = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, "bo", label="Training accuracy")
plt.plot(epochs, val_acc, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```



1.2.7 Evaluating on the test set

- Test acc: 0.669 compared to 0.66 of the RNN model

```
[19]: print(f"Test acc: {transformer.evaluate(test_ds)[1]:.3f}")
```

```
279/279 [=====] - 7s 27ms/step - loss: 2.4322 -  
accuracy: 0.6690  
Test acc: 0.669
```

1.2.8 Saving the trained Transformer model

```
[20]: # Hide the warning messages  
import logging  
  
tf.get_logger().setLevel(logging.ERROR)  
  
import warnings  
warnings.filterwarnings(  
    "ignore",  
    message="You are saving your model as an HDF5 file.*"  
)  
  
transformer.save("translation_with_Transformer.h5")
```

1.2.9 Buiding a Tokenizer

```
[21]: from tensorflow.keras.preprocessing.text import Tokenizer  
  
tokenizer = Tokenizer(num_words=vocab_size, oov_token="<UNK>")  
tokenizer.fit_on_texts(train_english_texts) # Use your original training_  
↳ dataset  
  
print("Tokenizer rebuilt successfully!")
```

Tokenizer rebuilt successfully!

1.2.10 Inspecting some test pairs

```
[22]: test_english_texts = [pair[0] for pair in test_pairs]  
  
print("Sample training sentence:", train_english_texts[20])  
print("Tokenized training sentence:", tokenizer.  
↳ texts_to_sequences([train_english_texts[20]]))  
  
print("Sample test sentence:", test_english_texts[25])  
print("Tokenized test sentence:", tokenizer.  
↳ texts_to_sequences([test_english_texts[25]]))
```

Sample training sentence: I don't work on Sunday.
 Tokenized training sentence: [[3, 22, 109, 29, 675]]
 Sample test sentence: A new room was assigned to me.
 Tokenized test sentence: [[7, 130, 131, 14, 6657, 4, 18]]

1.2.11 Translating new sentences with the Transformer model

```
[23]: spa_vocab = target_vectorization.get_vocabulary()
spa_index_lookup = dict(zip(range(len(spa_vocab)), spa_vocab))
max_decoded_sentence_length = 20

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization(
            [decoded_sentence])[:, :-1]
        predictions = transformer(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

test_eng_texts = [pair[0] for pair in test_pairs]
for _ in range(20):
    input_sentence = random.choice(test_eng_texts)
    print("-")
    print(input_sentence)
    print(decode_sequence(input_sentence))
```

```
-
The engine is purring.
[start] el [UNK] está bajando [end]
-
Bears like honey.
[start] a a los le les gusta la miel [end]
-
Is it a trap?
[start] eso fue una trampa [end]
-
It's very big.
[start] es muy grande [end]
-
Tom can't say for sure when Mary will arrive.
[start] tom no puede decir por cuándo llegue mary [end]
```

-
 She's a real gossip.
 [start] ella es un [UNK] [end]
 -
 Please answer all the questions.
 [start] por favor respuesta las preguntas [end]
 -
 How many kinds are there?
 [start] cuántos tipos de él hay [end]
 -
 They lost everything.
 [start] se perdió todo [end]
 -
 Tom and Mary were speaking in French, but switched to English when John entered the room.
 [start] tom y mary estaban hablando en francés pero al inglés le gusta john entró a la habitación [end]
 -
 I ran.
 [start] quedé [UNK] [end]
 -
 She advised him not to drive too fast.
 [start] le aconsejó que no [UNK] demasiado tom [end]
 -
 Have you bought your Christmas gifts yet?
 [start] has comprado a tus entradas ya tu [UNK] [end]
 -
 I seldom speak to him.
 [start] yo también le sé hablar con él [end]
 -
 I will get in touch with you.
 [start] me [UNK] con usted [end]
 -
 I know you must have a million questions.
 [start] sé que tienes que hacer algunas preguntas [end]
 -
 I'd never betray Tom.
 [start] yo también le daré un discurso a tom [end]
 -
 You came out of the room.
 [start] entró usted la habitación [end]
 -
 They went fishing.
 [start] se fue a pescar [end]
 -
 That's pretty lame.
 [start] eso es muy [UNK] [end]